

DeepInverse tutorial @ MICCAI 2026

Andrew Wang^{1,2}, *maintainer of DeepInverse*

¹*Blur Labs, Paris*

²*University of Edinburgh, Edinburgh*

27/09/26



Tutorial presented at MICCAI, Sunday 27th Sept 2026, Strasbourg, France.

Today's agenda

1. What is DeepInverse?
2. What is image reconstruction?
3. Why DeepInverse?
4. Overview of DeepInverse
5. Workflows for scientists and practitioners
6. On LLMs, docs and community
7. 🙌 Hands-on: live quickstart demo
8. Examples on real data
9. 🙌 Hands-on exercise, hacking + live support

Feel free to ask
questions at
any time!

What is DeepInverse?

DeepInverse is... an open-source library of research software

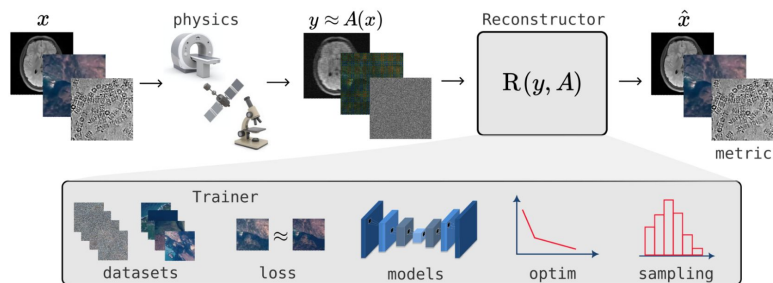


[Quickstart](#) [Examples](#) [User Guide](#) [API](#) [Benchmarks](#) [More](#) ▾

Q Search



DeepInverse: a Python library for imaging with deep learning

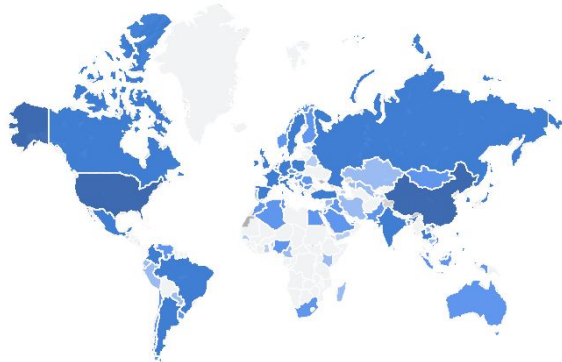


3

First of all, DeepInverse is a software project: it's an open-source library of research software, created and maintained by the community.

What is DeepInverse?

DeepInverse is... a community of users: academic researchers, industry practitioners, imaging professionals...



Global quarterly usage

COUNTRY	ACTIVE USERS
China	3.4k ↑754....
United States	1.2k ↑68.6%
Singapore	743 ↑1,38...
France	532 ↑17.4%
United Kingdom	257 ↑0.8%
Germany	214 ↑35.4%
Netherlands	123 ↑68.5%



Secondly, and perhaps more importantly, DeepInverse is a community of human users, including many experts in imaging, from researchers who develop new algorithms to practitioners who use these algorithms in industry, and professionals such as image analysts or radiologists. Thanks for coming to the tutorial and joining the community!

What is DeepInverse?

DeepInverse is... a community of contributors



5



Inria



ENS

PSL



EPFL



Thirdly, and my favourite, DeepInverse is a community of passionate contributors. These are researchers or users like you who see the benefit of contributing for disseminating their research and learning from other experts in AI or various imaging modalities. Here are some pictures of our latest hackathon in France in August 2026.

What is image reconstruction?

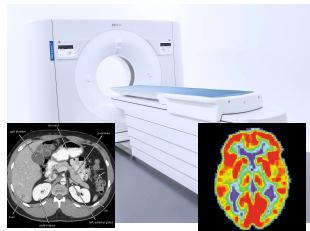
Solve the inverse problem: $\mathbf{y} = N(A(\mathbf{x}))$

Unifies and underpins every medical imaging modality

MRI



CT, nuclear medicine



ultrasound



optical imaging



 Deep Inverse

6

Every imaging modality is underpinned by the same equation. Let $\mathbf{x}$ be the image you're trying to obtain, but $\mathbf{y}$ is the measurements acquired by the scanner. $\mathbf{A}$ models a physical *forward operator* which takes into account the physics and geometry of the system, whereas $\mathbf{N}$ models random noise such as sensor readout or shot noise. For example, in MRI, the scanner $\mathbf{A}$ models a Fourier transform and $\mathbf{y}$ is the frequency measurements. In transmission or emission tomography, $\mathbf{A}$ models taking line integrals at various angles, so $\mathbf{y}$ is the detector response at every angle; emission tomography is characterised by very low photon-count, modelled by a higher Poisson gain in $\mathbf{N}$. In ultrasound, we can write the acquisition in a very similar way, using far fewer angles. Finally, in optical imaging, cameras take RAW measurements $\mathbf{y}$ and we still need to better reconstruct visual images, where $\mathbf{A}$ models lens blur, sensor mosaicing, or surgical motion.

Writing each modality in the inverse problem framework allows us to start using the same tools to solve different problems, meaning that we don't have to reinvent diffusion models or something for each modality.

Why DeepInverse?

There are huge unexplored synergies between imaging domains.

Image reconstruction and DL algorithms should be developed in a common framework.

Open-source software can close these gaps.

The collage shows four software interfaces: **parallelproj** (documentation for 2.0.2), **ASTRA Toolbox** (MATLAB Ultrasound Toolbox), **MUST** (MATLAB Ultrasound Toolbox), and **MRpro** (MATLAB MRI Toolbox). An arrow points from the **MRpro** API list to a code snippet in the **DeepInverse** interface.

parallelproj 2.0.2 documentation

Q, Search

Emission tomography reconstruction algorithms (sinogram data)

Convergence comparison: MLEM vs OSEM vs SVRG

Convergence comparison: SGD vs SVRG with logcosh regularization

PDHG and SPDHG for PET reconstruction with a directional TV prior

ASTRA Toolbox

Search docs

News

Downloads

Documentation

Algorithms

2D CPU Algorithms

FP

BP

FBP

SIRT

SART

ART

CGLS

2D GPU Algorithms

MUST

MATLAB Ultrasound Toolbox

beamform

das

Delay-and-sum of RF and IQ signals

das3

3-D delay-and-sum of RF and IQ signals

dasmx

Delay-and-sum matrix for beamforming with sparse matrix-vector multiplication

dasmx3

3-D delay-and-sum matrix for beamforming with sparse matrix-vector multiplication

MRpro

Search docs

API

mrpro.algorithms

- mrpro.algorithms.prowhiten_kspace
- mrpro.algorithms.total_variation_denoising
- mrpro.algorithms.varimax
- mrpro.algorithms.com
- mrpro.algorithms.dcf
- mrpro.algorithms.optimizers
- mrpro.algorithms.reconstruction
 - mrpro.algorithms.reconstruction.DirectReconstruction
 - mrpro.algorithms.reconstruction.IterativeSENSEReconstruction
 - mrpro.algorithms.reconstruction.Reconstruction
 - mrpro.algorithms.reconstruction.RegularizedIterativeSENSEReconstruction
 - mrpro.algorithms.reconstruction.TotalVariationRegularizedReconstruction

DeepInverse

Operators

The library provides a large variety of imaging operators `physics`: modelling $N(A(\cdot))$, which can simulate the observation process:

```
x = load_image()
y = physics(x) # simulate observation
```

Reconstruction

In order to recover an image from its measurements, the library provides many reconstruction methods $\hat{x} = R(y, A)$, which often leverage knowledge of the acquisition `physics`. Given a restoration model `model`, the reconstruction is therefore provided as

```
x_hat = model(y, physics) # reconstruct sig
```

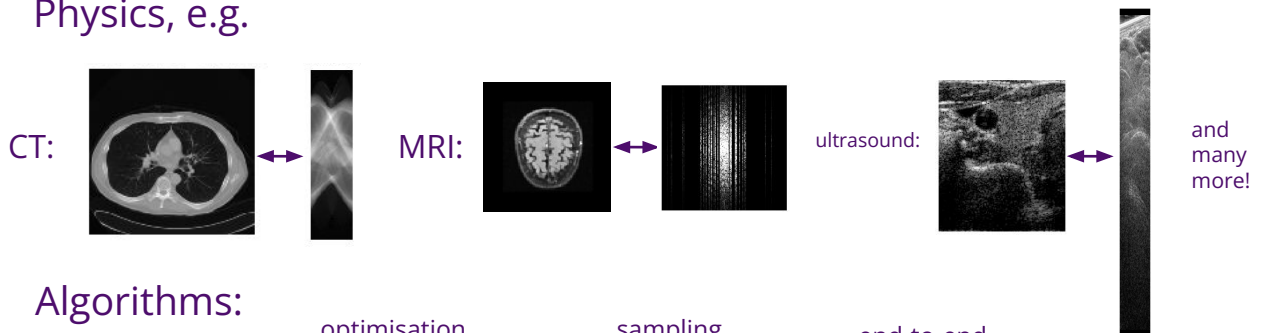
7

As we've seen, there are benefits from modelling problems in the same framework. Now with the proliferation of DL algorithms for image reconstruction, it will be wasteful to redevelop and reimplement these methods for every modality, slowing down research and translation. However, traditionally these modalities have developed software independently. For example, each library listed here have super useful modality-specific projection operators: **parallelproj** for PET projectors, **ASTRA** for arbitrary CT projectors, **MUST** for ultrasound simulation, **mri-nufft** for non-Cartesian MRI or **MRPro** for MRI. In each of these libraries, many identical basic reconstruction algorithms are implemented each time: gradient descent, conjugate gradient least squares, filtered/density-compensated backprojections, TV regularisation... This makes historical sense, but model-based deep learning, sampling, and end-to-end neural networks have advanced way beyond these methods, and crucially have advanced often **outside** of these subfields: for example, diffusion models originated in natural imaging. Attempting to reimplement these in each specific library will inevitably lead to gaps wrt. SotA.

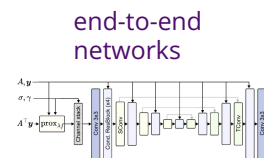
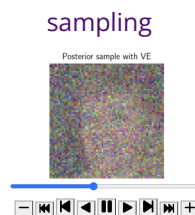
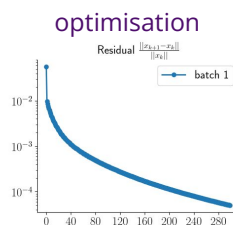
DeepInverse solves this by providing an abstracted framework for image reconstruction, abstracting any operator into ``physics`` objects, and any reconstruction algorithm into ``model`` objects.

Overview of DeepInverse

Physics, e.g.



Algorithms:



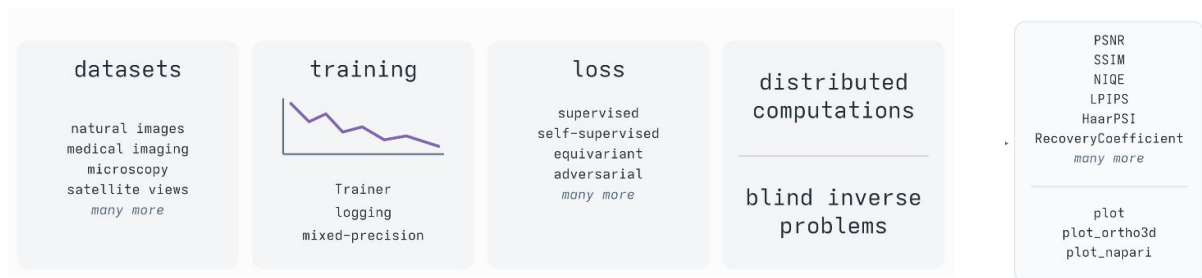
8

 Deep Inverse

As mentioned, physics link images to measurements and in the linear-case, there is a bidirectional link provided by the forward and backward projectors. Algorithms today fall into multiple categories, including mathematical optimisation, sampling (including diffusion models, flow-matching and MCMC), and end-to-end networks including unrolled networks or other modern neural network architectures. **Crucially**, note that DeepInverse is a framework that evolves every day, not just a collection of algorithms, and the community of experts continually contributes the latest SotA methods (e.g. INRs, Gaussian splatting...). What I've presented here is only a snapshot!

Overview of DeepInverse

- Also lots of useful tooling



I lied: there is a third half of DeepInverse, which is all the tooling you need to perform experiments for image reconstruction with ML. This includes public datasets for training and testing, a modular trainer, many loss functions for training models with and without ground truth, tools for distributing heavy physics (e.g. 3D projectors) across GPUs, tools for dealing with blind inverse problems, and many useful image quality metrics, from generic full-reference metrics to modality-specific no-reference metrics, such as NIQE for natural images or the lesion recovery coefficient for emission tomography.

Overview of DeepInverse

- Digression: goes way beyond medical imaging, but don't let that stop you...

Inpainting, Blur, Downsampling, Demosaicing, SpaceVaryingBlur, Pansharpen, CompressiveSpectralImaging, CompressedSensing, RadioInterferometry, SinglePhotonLidar, PhaseRetrieval, Ptychography, Scattering...

A small digression: DeepInverse goes way beyond medical imaging, as the inverse problem framework is everywhere! I hope that you can be inspired by applications of imaging and methods proposed elsewhere, since there are many common problems, e.g. RadioInterferometry for astronomy has similar physics to non-Cartesian MRI. See the DeepInverse docs for the full list:

https://deepinv.org/user_guide/physics/physics.html

Workflows

- Workflows for algorithm/AI researchers:
 - Test your own algorithms with different inverse problems
- Workflow for practitioners & clinicians:
 - Test SotA algorithms on your own scanner data
- Workflows for scientists/physicists: own physics
 - Test SotA algorithms on your novel scanner physics

The DeepInverse framework allows different workflows for different users. For example, if you are an AI researcher, you can test your new algorithms that you implement themselves, on various existing inverse problems implemented in DeepInverse. Secondly, if you're a practitioner or clinician, you can use off-the-shelf SotA algorithms implemented in DeepInverse with your own data that you acquire and load. Finally, if you've developed a new scanner or physical model, you can test your new physics with existing datasets and algorithms.

On LLMs, docs and community

- DeepInverse is a library by humans and for humans
- Research software serves to teach - see our docs!
- For your experiments, LLMs/agents are great!
- LLMs/agents work even better if...
 - They make use of existing open-source tools, rather than rewriting -> why we've carefully designed DeepInverse.
 - You can write good prompts based on fundamental understanding -> why we write human docs.



DeepInverse is a library designed by humans, for humans. Why do we place so much importance on this? Firstly, research software serves to teach, and we know that when we write code and documentation pedagogically, we both give enable new researchers to develop fundamental understanding about image reconstruction, and inspire new directions of research. Everyone knows the feeling of trying to understand an unfamiliar library, and having to wade through slop. We've spent so much time writing human-readable documentation and won a prize in 2024.

We also make it easy for you to use LLM agents and DeepInverse to accelerate your experiments and research. LLM agents can really help but only if you have a good setup. Firstly, we've carefully designed DeepInverse to be an opinionated, abstracted framework that allows fast prototyping for both humans and agents. Secondly, your agent is only as good as your prompt, and you can only write good prompts based on fundamental understanding by reading the human documentation.

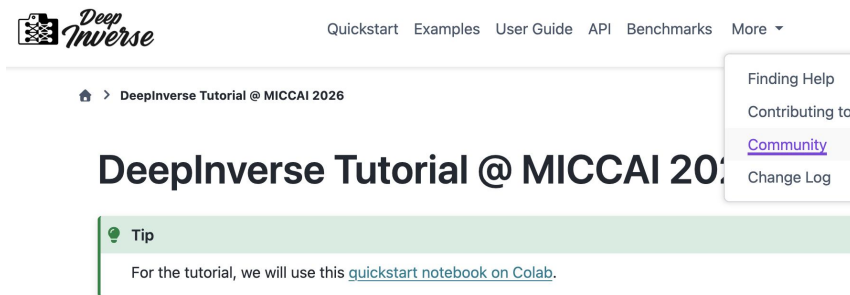


Hands-on: live quickstart demo!



<https://colab.research.google.com/drive/1Cewh2PmSNCf8JPARGzIEz2eM-mUNMQSe?usp=sharing>

1. Navigate to <https://deepinv.org>
2. Click More→Community→MICCAI tutorial→Tip



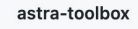
13



Get started with a quickstart demo for medical imaging that I designed for this MICCAI tutorial! Navigate to the Colab link, choose a GPU runtime (CPU will be fine too, just a bit slower) and step through the cells and fill in the blanks.

Examples on real data

- Accelerated MRI reconstruction with pretrained models
- Sparse-view & low-dose CT reconstruction benchmark
- Low-dose PET reconstruction
- Ultrafast ultrasound reconstruction
- Non-Cartesian MRI reconstruction
- ...
- All available here: https://deepinv.org/auto_examples



The quickstart demo of the previous slide showed how to simulate measurements with physics and reconstruct them with algorithms. Here are some real-data examples that we have in DeepInverse that show you how to load raw data and reconstruct them with existing algorithms, for MRI, CT, PET, ultrasound etc. Here is a list of 5 examples, but please go on the **examples**, filter by application, and browse! https://deepinv.org/auto_examples



Reconstruct undersampled k-space for cardiac and brain MRI

This demo reconstructs undersampled k-space data for 2D cardiac and brain MRI on:

- single-coil cardiac cine MRI from [CMRxRecon](#) train set (`deepinv.datasets.CMRxReconSliceDataset`) ^[1];
- 12-coil brain k-space from [Calgary-Campinas](#) test set (`deepinv.datasets.CalgarySliceDataset`) ^[2];
- 16-coil brain k-space from [FastMRI](#) test set (`deepinv.datasets.FastMRISliceDataset`) ^[3].

We demonstrate pretrained models:

- Joint-ICNet ^[4], pretrained on Calgary data, from [DIRECT](#);
- vSHARP ^{[5][6]}, pretrained on fastMRI brain, knee, prostate, and CMRxRecon cardiac data as the [UNIFORM](#) model, from [DIRECT](#);
- [RAM](#) ^[7], pretrained on natural images, abdominal CT and knee MRI.

Note

This example requires [DIRECT](#) (Netherlands Cancer Institute) and Python >=3.12. Install with `pip install deepinv[direct]`.

```
import torch
import deepinv as dinv
```



In-vivo ultrafast ultrasound reconstruction with Plug-and-Play

This example shows how to use `deepinv.physics.UltrasoundPlaneWave` to reconstructs an in-vivo carotid acquisition of the [EPFL LTSS ultrafast ultrasound dataset](#) from raw RF ultrasound data.

In ultrafast ultrasound, data is acquired with plane-waves i.e. unfocussed transmits. Typically, images are reconstructed with the adjoint i.e. delay-and-sum (DAS) beamforming. However, DAS with 1 or very few plane-waves is generally low quality, so one often uses many `n_angles >> 1` plane-waves. This is called coherent plane-wave compounding (CPWC), but of course increases the acquisition time by a factor of `n_angles`. Instead, we can therefore use more advanced image reconstruction techniques to reconstruct from very few plane-waves.

In this example, we demonstrate:

1. **DAS with 87 plane-waves** (CPWC): the adjoint using all transmitted angles, giving reference image quality.
2. **DAS with only 1 plane-wave**: the adjoint of the operator restricted to 1 plane-wave transmit. We expect low quality with high amount of sidelobes and low SNR.
3. **Least squares with 1 plane-wave** with `A_dagger` using conjugate gradient.
4. **Plug-and-Play with 1 plane-wave** with `proximal gradient descent` with a `PnP` prior using wavelet and BM3D denoisers.



Reconstruct real CT sinograms with the 2DeteCT benchmark

We demonstrate image reconstruction of acquired CT projection data in sparse-view, limited-angle and low-dose CT acquisition scenarios.

The data is taken from the 2DeteCT benchmark [\[1\]](#) and dataset [\[2\]](#), which is an industrial CT dataset of various materials acquired using a proprietary scanner from CWI (i.e. sinogram-to-image). The setup is matched exactly to Kiss *et al.* [\[1\]](#), such that you can compare DeepInverse image reconstruction methods with the values reported in Kiss *et al.* [\[1\]](#).

Note

This example requires `astra`. Install it with instructions from [their docs](#) using `pip`, `conda` or `conda-forge`, e.g. `pip install astra-toolbox`. Note that `astra` only supports CUDA.

This example also requires `tifffile`. Install it with `pip install tifffile`.

```
import deepinv as dinv
import torch
```



3D PET reconstruction with the Brainweb dataset

This example reconstructs a volume from the BrainWeb [casperdc/brainweb](https://casperdc.github.io/brainweb) positron emission tomography (PET) dataset. We compare standard PET reconstruction algorithms with methods that support penalized objective functions.

OSEM and BSREM minimize the Poisson negative log-likelihood

$$f(x) = \mathbf{1}^T(Ax + b) - y^T \log(Ax + b),$$

and BSREM additionally uses the Relative Difference Prior (RDP) ([deepinv.optim.RDP](#)) as g_σ in $f(x) + \lambda g_\sigma(x)$. RDP favors sharp transitions in reconstructed images and adapts to the local signal level, which is particularly useful for emission tomography where the dynamic range can be large. We also demonstrate general-purpose gradient descent with a least-squares objective.

Note

This is a large 3D example and is intended to run on a CUDA-capable machine. It requires the `brainweb` and `parallelproj` packages. Install them with `pip install brainweb parallelproj`.



🏠 > Examples > Models > Reconstruct prospectively-undersampled raw multicoil MRI

Reconstruct prospectively-undersampled raw multicoil MRI

This example reconstructs real prospectively undersampled multicoil brain k-space from Yu *et al.*^[1].

This demonstrates the performance of reconstruction algorithms in a deployment scenario rather than a typical simulated (retrospective) scenario. Here, it is impossible to compute full-reference metrics, since fully-sampled ground-truth does not exist.

We compare two types of reconstruction:

- Deep learning, using a pretrained general model `deepinv.models.RAM` from Terris *et al.*^[2]
- Compressed sensing, using FISTA with a wavelet prior^[3]

The data is stored in the raw ISMRMRD format.

Note

This example requires `ismrmrd` to load the data. Install it with `pip install ismrmrd`.

```
import torch
import deepinv as dinv

device = dinv.utils.get_device()
```





🏠 > Examples > External Libraries

> Reconstruct accelerated non-Cartesian breast MRI acquisition data

Reconstruct accelerated non-Cartesian breast MRI acquisition data

This example reconstructs raw non-Cartesian multicoil kspace data from the FastMRI breast dataset [\[1\]](#), for mammography.

The data was acquired using radial sampling, with 288 spokes. We compare image reconstruction using all spokes vs. with 72 spokes (i.e. 4x acceleration). We model the 2D non-uniform FFT physics with [deepinv.physics.NonCartesianMRI](#), which uses [MRI-NUFFT](#).

Note

This example requires the `mri-nufft` library to model the physics. Install with `pip install mri-nufft[finufft]` (CPU or MPS) or `pip install mri-nufft[cufinufft]` (GPU).

```
import torch
import deepinv as dinv

device = dinv.utils.get_device()
```





🏠 > Examples > Self-Supervised Learning
> Scan-specific zero-shot SSDU for MRI

Scan-specific zero-shot SSDU for MRI

We demonstrate scan-specific self-supervised learning, that is, learning to reconstruct MRI scans from a single accelerated sample without ground truth.

Here, we demonstrate fine-tuning a pretrained model ([deepinv.models.RAM](#)) ^[1] with the [weighted SSDU](#) loss ^{[2][3]}. However, note that any of the [self-supervised losses](#) can be used to do this with varying performance ^[4]. For example see the [example using Equivariant Imaging](#) ^[5].

Note that, if more data is available, better results can be obtained by fine-tuning on more samples!

```
import torch
import deepinv as dinv
```





🏠 > Examples > Self-Supervised Learning > Low-field MRI denoising without ground truth

Low-field MRI denoising without ground truth

We demonstrate self-supervised (blind) denoising of a low-field MRI scan without ground truth data.

In low-field MRI, images have high noise due to the fixed permanent magnet, and acquiring clean reference images is often impossible. One could average the images over multiple repetitions, but if there's any patient motion, the image would be blurry.

Here, we fine-tune the Reconstruct Anything Model ([deepinv.models.RAM](#)) ^[1] on a single noisy scan using the self-supervised [Reconstructed2Reconstructed Loss](#) ^[2]. Play around with different self-supervised denoising losses (see [Self-Supervised Learning](#)) and models!

Note that, if more data is available, better results can be obtained by fine-tuning on more samples.

```
import torch
import deepinv as dinv
```





🏠 > Examples > Self-Supervised Learning
> Ultrasound despeckling from B-mode images

Ultrasound despeckling from B-mode images

This example despeckles real clinical ultrasound B-mode images with Speckle2Self [\[1\]](#), a model pretrained without clean reference images.

We compare its performance against a classical BM3D `deepinv.models.BM3D` baseline.

We use B-mode images from the carotid dataset acquired in Speckle2Self.

Ultrasound B-mode images are corrupted by speckle, a granular noise pattern that arises from interference between backscattered echoes. Speckle lowers contrast and hides fine tissue structure, which makes clinical interpretation harder.





🏠 > Examples > Optimization > Multispectral demosaicing from raw sensor data

Multispectral demosaicing from raw sensor data

This example reconstructs a full-resolution multispectral image from raw snapshot mosaiced measurements using various reconstruction algorithms.

Snapshot multispectral cameras cover the sensor with a mosaic of spectral filters (multispectral filter array), so that each pixel records only one band. Demosaicing recovers every band at the full sensor resolution.

Demosaicing is a critical part of any image signal processing (ISP) pipeline, and using better algorithms results in higher quality images for downstream tasks.

We use raw oral-tissue data from the MODID dataset [\[1\]](#) acquired in-vivo for screening of oral diseases such as oral squamous cell carcinoma. See [original data source](#). The data is unprocessed from an imec CMV2K-SSM4x4-VIS CMOS sensor covering 16 bands from 460 to 600 nm before any corrections. The sensor uses a sequential 4x4 MSFA i.e. 16 bands.

```
import torch
import deepinv as dinv
```



Add a star on GitHub!

Give us a star on GitHub! <https://github.com/deepinv/deepinv>

Let's keep in touch!



<https://forms.gle/cBkxhj1EMGc5Tnn28>

20



Want to reach out and collaborate? Please fill in the form and I'll reach out!

👐 Hands-on hacking

- Option 1: complete MRI training exercise in the quickstart notebook
- Option 2: work on whatever you want!
 - Take inspiration from the demos
 - Try your own recon algos on different inverse problems
 - Try integrating different deepinv solvers into your existing experiments
 - Replace x/y with your own/other public data
- The more you engage, the more you benefit
- We are here until 12h30 to provide live support! (me and Vicky):
- Don't worry if you can't finish something - we can continue to support you



Have a play yourself! You have the exercise in the existing notebook, or, for example, you can take inspiration from existing demos and try using your own recon algorithms or own data to play around. If you would like any support or a chat, please join the DeepInverse Discord <https://discord.gg/qBqY5jKw3p>, open an issue on GitHub, <https://github.com/deepinv/deepinv/issues> or email me andrew.wang@ed.ac.uk.